

AeroOps — Objection-Handling Guide

For: anyone in a sales conversation with a flight school, club, university, FBO, maintenance shop, or corporate flight department.

The one rule: never overclaim. A six-figure buyer tests claims, and one caught exaggeration discounts everything else you said. When a capability is on the roadmap, say **"on the roadmap"** and show the seam. Honesty is not a weakness in this deck — it's the differentiator, because our whole pitch is a *system you can trust*.

How to use each entry: understand the *real* concern under the objection, respond in 2–4 sentences grounded in a capability that actually ships, then **show the proof** in the live demo.

1. "We already use Flight Circle / Flight Schedule Pro."

The real concern: switching cost and risk. They've invested time; the incumbent works "well enough."

Response: > "Good — then you already know what a scheduler feels like, and we're not asking you to relearn that. The question is what happens *after* the flight: does your current tool turn a closeout into an invoice, a logbook entry, and a squawk in one motion, or do those live in separate places where things fall through the cracks? That gap is where un-billed flights and lapsed inspections come from. And you don't have to guess — our Import Center reads Flight Circle and FSP exports directly, so you can trial us on your real data and roll it back if you don't like it."

Proof to show: The dispatch closeout (`/dispatch`) generating the invoice on `/billing` and the squawk on `/maintenance` without touching either — then `/import` showing the Flight Circle / FSP source presets and the rollback.

2. "It costs too much."

The real concern: they can't see the ROI, or they're comparing a price to \$0 (the spreadsheet).

Response: > "The honest frame isn't features versus price — it's leakage versus price. One dual instructional flight a week that flies and never gets billed is more revenue than a subscription costs, and our closeout makes that specific leak *structurally impossible* — the flight can't close without producing the invoice. Add the inspection you don't overrun and the checkride you don't lose to a missing endorsement, and the tool pays for itself before it's convenient."

Proof to show: The atomic closeout on `/dispatch` (invoice + meters + hours + balance in one transaction), and the `/executive` view where revenue and AR aging are visible at a glance.

3. "Our instructors won't learn a new system."

The real concern: change fatigue and lost billable time during a rollout.

Response: > "Your instructors learn exactly one screen: dispatch. Release a flight with a checklist, close it out with the Hobbs reading — that's their entire interaction, and it's fewer steps than juggling a scheduling app, a paper dispatch binder, and a separate invoice. Everything downstream — billing, the logbook hours, the squawk — happens off that one closeout. And because students self-serve their own schedule and progress, your CFIs actually field *fewer* interruptions, not more."

Proof to show: The single closeout action on `/dispatch`, then the student's self-service world (log in as `student@aerops.demo`) so they see the CFI isn't the help desk.

4. "We don't want another piece of software."

The real concern: tool sprawl — they already stitch together three apps and a spreadsheet, and dread a fourth.

Response: > "That's exactly the problem we exist to end — AeroOps is meant to *remove* software, not add to the pile. Scheduling, dispatch, maintenance, training, and billing are one data model here, so the schedule already knows the airplane is grounded and the invoice already

Proof to show: The dashboard's single "Needs attention" triage (`/dashboard`) pulling squawks, maintenance, checkrides, and activity from one system — and the closeout proving those surfaces share data, not sync it.

5. "We've always done it this way."

The real concern: the current process feels safe because it's known; change feels like risk to a working operation.

Response: > "Respect for that — a process that's kept you flying safely is worth something, and we're not asking you to change how you *operate*, only where the record lives. The risk in 'the way it's always been done' is the information that exists but isn't connected: the squawk that didn't reach the scheduler, the endorsement nobody checked before the checkride. AeroOps makes the *connected* version the easy version — the safe, compliant path becomes the path of least resistance, not extra work."

Proof to show: The airworthiness gate blocking a release on `/dispatch` (the system enforcing what a person used to have to remember), and the audit trail underneath every action.

6. "What about our existing data?"

The real concern: the incumbent is holding their data hostage; migration horror stories.

Response: > "This is the fear we built directly into the product, because it's the number-one reason people stay on a tool they've outgrown. The Import Center takes exports from Flight Circle, Flight Schedule Pro, FlightLogger, QuickBooks and more — students, aircraft with Hobbs/Tach and rates, schedules, maintenance, invoices, and balances. You map the columns once, run a *test* import that uses the identical code path but rolls itself back so the preview is the truth, then commit — and every record we create is tracked, so a full rollback is one action. Nothing is silent: any row that can't import comes back with a reason and a downloadable failed-row file."

Proof to show: `/import` — the source presets, the test/commit/rollback flow, and the failed-row handling.

(Honesty note: importing endorsements, training records, and document metadata is on the roadmap; the core operational and financial data types ship today.)

7. "Is our data secure? Who can see what?"

The real concern: multi-tenant anxiety (can another org see us?) and internal access control (can a student see the P&L?).

Response: > "Two separate guarantees, both enforced by construction. Tenant isolation: your organization's scope comes from the authenticated session, never from the browser, so one org's data cannot be queried by another — it's not a policy, it's how every query is built. Internal access: the app authorizes on *permissions*, not job titles, on every page and every API route — a student literally cannot load the billing route, and a dispatcher sees operations but not the org's finances. And every change is written to an immutable audit trail: who did what, when, never rewritten."

Proof to show: Log in as `student@aerops.demo` vs. `admin@aerops.demo` side by side — the student's dedicated workspace with no finance, fleet, or other-student data — then reference the audit trail behind the closeout you ran earlier.

8. "What if you go out of business?"

The real concern: vendor risk — they don't want to be stranded with their operation trapped in a dead product.

Response: > "Fair question, and the answer is that your data is never hostage to us the way it is to your current vendor — which is exactly why we built export and rollback in from day one. You can pull your data out in clean, structured form, and the audit trail and records are yours. We'd rather earn the renewal every year by being the system you *want* to run on than lock you in by making leaving painful. The honest posture is: we're an early-stage company in a controlled beta, we tell you that plainly, and we de-risk it by never trapping your data."

Proof to show: The Import Center's export/rollback mechanics on `/import` (the same machinery that gets data *in* respects getting it *out*), and be candid about company stage — buyers trust the vendor who volunteers the risk.

Quick-reference: objection → anchor capability

Objection	Anchor (all shipped unless noted)
Already use a competitor	Import Center with rollback + dispatch + invoice atomicity

Objection	Anchor (all shipped unless noted)
Costs too much	Closeout makes billing leakage structurally impossible
Instructors won't learn it	One screen (dispatch); student self-service reduces CFI load
Don't want more software	One data model replacing 3 apps + a spreadsheet
Always done it this way	Airworthiness gate + audit trail enforce the safe path
Existing data	Import Center: map → test (rolled back) → commit → rollback manifest
Security / who sees what	Session-scoped tenant isolation + permission-gated RBAC + audit trail
What if you fold	Structured export + rollback; data never held hostage; honest about stage

Roadmap items to never misrepresent: customer card payments online (Stripe Connect), live METAR/TAF weather (simulated today), transactional email (invites shown in-app today), document file uploads (metadata-only today), cross-location/cohort reporting roll-ups. Say **"on the roadmap"** — and show the seam where it plugs in.